OBL2-OS

1. Synchronisation

1. Communication between isolated processes

Example

A shell pipeline such as `ls | grep .c` is the simplest example: `ls` produces a listing and `grep` consumes it, yet the two run as separate processes with separate address spaces. A more general example is a client and a server over a TCP socket - a browser requesting a page from a web server - or a GUI application handing a framebuffer to the display server through shared memory.

How it works

The kernel is the only entity that can see both address spaces, so every IPC mechanism is mediated by it through system calls. The kernel creates an IPC object - a pipe buffer, a message queue, a socket, or a shared physical page - and hands each process a handle, which is a a file descriptor or an identifier. Data then moves either by copying through the kernel or by mapping the same physical frames into both page tables.

For the pipe: `pipe()` returns two file descriptors referring to one kernel buffer. `ls` calls `write()`, which traps into the kernel and copies bytes from `ls`'s user space into the buffer; `grep` calls `read()`, which copies them out into `grep`'s own address space. Neither process ever holds a pointer into the other's memory - the copy is what preserves isolation. The kernel also supplies the synchronization: a reader on an empty pipe blocks until data arrives, a writer on a full pipe blocks until space is freed, and `read()` returns 0 (EOF) once all writers have closed their ends.

```
  process A                      kernel                      process B
user buffer  --write()-->  [ pipe buffer ]  --read()-->  user buffer
             (copy in)                      (copy out)
```

Resulting problems from inter-process communication

Inter-process communication is not free, and most of its problems follow from the fact that the kernel, not the processes, owns the channel. Any shared state - a shared memory segment, a pipe with several writers - needs a protocol of its own, because without one producer/consumer ordering breaks,

and data is lost or duplicated; the same missing protocol lets two processes deadlock, each holding one lock and requesting the other, or a writer block on a full pipe while the reader waits for a message that will never come.

IPC is also expensive. Every copy through the kernel and every block/wake-up is a context switch, which makes IPC orders of magnitude slower than for example a function call. This is exactly why threads are preferred when the data is shared. It is also exposed as an attack surface as well, so a process must authenticate its peer, distrust message contents, and avoid leaking data through inherited descriptors, since shared memory in particular bypasses all kernel checks once the mapping exists.

Finally, the channel imposes framing and lifecycle obligations of its own. Stream channels have no message boundaries, so a read() may return half a message or two at once, a crashed peer can leave the survivor blocked forever unless it detects EOF or uses timeouts, and both sides must agree on a name that does not collide.

2. What is a critical region?

Definition

A critical region is a piece of code that accesses a shared resource - such as a shared variable, a file, a device, or a data structure - and must therefore never be executed by more than one process or thread at a time. The surrounding protocol is usually described as an entry section that requests permission, the critical section itself, an exit section that releases it, and a remainder section. A correct solution of the protocol must therefore satisfy: mutual exclusion (at most one process inside), progress (if the section is free, some waiting process must be able to enter, and the choice cannot be postponed indefinitely by processes in their remainder), and bounded waiting (no process is starved).

Can a process be interrupted while in a critical region?

Yes - and it usually will be. The critical region is a software convention; the CPU has no notion of it. A timer interrupt, a page fault, an I/O completion, or a pre-emptive context switch can occur at any instruction boundary, including in the middle of the region. That is precisely why the region must be protected by an atomic protocol rather than by hoping the code runs to completion: the mutual-exclusion guarantee comes from the lock, not from non-interruptibility.

The only way to actually prevent interruption is to disable interrupts, and that is a poor general solution which only works on a uniprocessor. On a multicore, masking interrupts on one core does nothing to stop another core from entering the same region. Even so, a long region with interrupts disabled

destroys responsiveness and can lose interrupts. Real systems therefore build locks from hardware atomic instructions and disable interrupts only for a few instructions inside the kernel.

3. Busy waiting versus blocking

Both are ways for a process to wait for a lock; they differ in what the process does with the CPU while it waits.

Busy waiting

Busy waiting (polling, spinning) keeps the process in the running state: it executes a tight loop that repeatedly tests the lock variable and tries to acquire it. The scheduler still considers it runnable, so it burns a full time slice doing nothing useful. The test must be an atomic read-modify-write, otherwise two waiters can both observe the lock as free. Its advantage is latency: there is no context switch, so if the lock is released within a few hundred nanoseconds, spinning is cheaper than the two context switches a block/wake-up pair would cost. Its disadvantages are wasted CPU, and the fact that on a single-core machine with a non-preemptive scheduler it deadlocks outright - the spinning process never yields, so the holder can never run to release the lock.

Blocking

Blocking (sleep/wake, wait/signal) gives up the CPU. The process makes a system call, the kernel moves it from running to blocked, removes it from the ready queue, and schedules someone else. When the holder releases the lock it performs a wake-up signal, which moves one waiter back to the ready queue. No CPU is wasted regardless of how long the wait is, and the scheme works on any number of cores. The price of this is overhead. Two context switches plus two kernel entries per acquisition is significant if the critical section is only a few instructions long.

Rule of thumb

The rule of thumb is therefore to spin when the expected wait is shorter than the cost of a context switch and there is a core available to spin on; block otherwise. Real implementations hedge - Linux futexes and Java's adaptive mutexes spin briefly and then block - and the kernel uses pure spin-locks only for very short critical sections, usually with pre-emption disabled.

4. What is a race condition?

A race condition is a situation in which the correctness of a program depends on the relative timing, the interleaving, of concurrent accesses to shared state, where at least one of those accesses is a write. The program has more than one possible outcome depending on who runs when, and some of

those outcomes are wrong. The classic mechanism is that a high-level operation is not atomic. `counter++` compiles to three instructions - load, add, store - and the scheduler can switch between any two of them.

```
// both threads execute this; expected 2, often 1
counter++;   // load counter -> add 1 -> store counter
```

If both threads load the same value before either stores, one increment is lost. The same pattern underlies check-then-act bugs, where a condition is tested and then acted on with a window in between; - Time-of-check to time-of-use (TOCTOU).

A real-world example is two people withdrawing from the same bank account at two ATMs at the same moment. The balance is 1000. Both machines read 1000, both verify that 800 can be withdrawn, and both write back 200. The account ends at 200 even though 1600 was paid out - the bank has lost 800. The same thing happens when two customers book the last airline seat, when two processes append to the same log file, or when two threads increment the same hit counter on a web server.

5. What is a spin-lock, and why and where is it used?

A spin-lock is a lock whose waiting behaviour is busy waiting. A thread that fails to acquire the lock loops, re-testing the lock word with an atomic instruction (test-and-set, exchange, or compare-and-swap) until the acquisition succeeds. It is the simplest mutual-exclusion primitive, and it needs only one shared word plus hardware atomicity.

It is used because it is cheap when contention is brief and because it works where blocking is impossible. If the critical section is shorter than the cost of blocking and waking a thread, ~roughly two context switches, spinning wins. The waiter acquires the lock almost immediately after release, with no kernel entry at all. More importantly, spin-locks are the only option in contexts that must not sleep. For example: inside an interrupt handler, inside the scheduler, while holding another lock that the blocking path itself needs, or in early boot before the scheduler exists.

6. Thread synchronisation on multicore; MCS and RCU

The issues.

On a multicore the cost of synchronization is dominated by the memory system rather than by the instructions themselves.

- **Cache coherence and contention.** A lock word lives in one cache line. Every atomic read-modify-write must take that line exclusively, invalidating it in every other core's cache, so a

contended lock bounces the line between cores ("cache-line ping-pong"). With P spinning cores, a naive test-and-set lock generates $O(P)$ coherence traffic per acquisition and gets slower as cores are added.

- **Memory ordering.** Compilers and CPUs reorder loads and stores. x86 provides total store order, ARM and POWER are weakly ordered, so synchronization must use acquire/release semantics or explicit fences; a lock that is correct on x86 can be wrong on ARM.
- **Scalability and fairness.** A single global lock serializes the whole system, and contention grows faster than linearly with core count. FIFO ordering, barging, and starvation all become design questions.
- **False sharing.** Two unrelated variables on the same cache line cause coherence traffic even though no data is shared.
- **NUMA.** Remote memory is far slower, so locks and the data they protect should stay node-local.
- **Blocking cost.** Context switches are expensive, so spinning is often preferable to blocking on multicore processors - which pushes designs toward the two algorithms below.
- **Correctness.** Races are nondeterministic and hard to reproduce; lock-free code adds ABA-problems and memory-reclamation hazards.

MCS lock

The MCS lock attacks the scalability problem of test-and-set spin-locks. Instead of having every waiter spin on the same word, it builds an explicit FIFO queue. Each thread owns a node, and acquisition atomically appends that node to the tail of the list. A thread then spins on the `locked` flag *in its own node*, which is a private cache line no other core touches, and the releasing thread writes only to its immediate successor's flag. Spinning therefore generates no coherence traffic at all, each acquisition costs $O(1)$ remote references instead of $O(P)$, and the queue gives FIFO fairness and bounded waiting for free. The costs are space (one node per thread, which must outlive the wait) and the requirement that a thread have a stable per-thread structure to enqueue.

The hardware mechanism at its heart is the atomic read-modify-write, specifically an atomic exchange, `xchg`, or compare-and-swap used once to splice the node onto the tail pointer. The local spin then relies on ordinary cache-coherent loads.

RCU

RCU (read-copy-update) attacks the opposite problem: read-mostly data, where readers vastly outnumber writers and a lock would make every reader pay for the mere possibility of a writer. Under RCU, readers take no lock and execute no atomic instruction at all - they simply dereference the pointer inside a read-side critical section. A writer never modifies data in place; it copies the structure, modifies the copy, and publishes it with a single atomic pointer store. Readers concurrent with the store see either the old version or the new one, never a torn mixture. The old version is freed only after

a grace period, once every reader that could still hold a reference has passed a quiescent state. The constraints are that read-side sections must not block or sleep, and that writers pay the copying and deferral cost.

The hardware mechanism at its heart is atomic word-sized load and store plus memory ordering. An aligned pointer load or store is indivisible in hardware, and release/acquire barriers ensure the new copy is fully initialised before the pointer is published. Notably, RCU's read side uses no read-modify-write at all, which is exactly why it scales: readers never write to shared memory, so they never invalidate each other's cache lines.

2. Deadlocks

1. Resource starvation versus deadlock

Starvation is indefinite postponement. This happens when a process is runnable and could in principle proceed, but the allocation policy keeps passing it over. Some examples are: a low-priority process in a priority scheduler, or a writer under a readers-preferring readers-writers lock. The system as a whole still makes progress, and the starved process would proceed if the competing processes finished. A deadlock on the other hand is a permanent standstill. A set of processes each hold a resource and wait for one held by another in a cycle, so none can ever proceed no matter how long we wait, and only outside intervention, for example killing a process or preempting its resources, breaks it. The distinction is therefore progress of the system versus progress of the individual, and a cycle versus no cycle. Starvation needs no circular wait and can occur with a single resource, whereas deadlock is by definition a circular wait.

2. The four necessary conditions for a deadlock

Coffman's four conditions are mutual exclusion, hold-and-wait, no preemption, and circular wait. All four must hold simultaneously for a deadlock to occur, so breaking any one of them prevents it.

Mutual exclusion is when a resource is non-shareable and held exclusively. This is inherent because it follows from the nature of the resource itself, such as a printer or a lock word, and not from OS policy. No preemption is when a resource is released only voluntarily by its holder. This is largely inherent as a printer cannot be snatched mid-job, nor a lock held inside a critical section, without corrupting state. Hold-and-wait is when a process holds one resource while requesting another. This is a policy choice because it disappears if a process must request all its resources at once. Circular wait is when a cycle of holders and requesters forms. This is a policy choice as it is only a structural consequence of request order, and it is removable by imposing a global ordering on acquisition.

That is exactly why prevention schemes (all-at-once allocation, resource ordering) target the latter two.

3. How an operating system detects a deadlock state

The OS detects a deadlock by building a wait-for graph where nodes are processes, and an edge $P_i \to P_j$ means P_i is blocked waiting for a resource that P_j holds. A cycle in that graph is a deadlock only when every resource has a single instance. With multiple instances per type, a cycle is necessary but not sufficient to deduce a deadlocked state, because a process outside the cycle may hold a spare instance of the same resource type and release it, satisfying one of the waiting requests. For example, if R_1 has two instances, one held by P_1 and one by P_3, and P_2 holds R_2 while waiting for R_1, the graph contains the cycle $P_1 \to R_2 \to P_2 \to R_1 \to P_1$, yet no deadlock exists: P_3 releases its instance, P_2 acquires it, finishes, and releases R_2 for P_1. The OS must therefore run the matrix reduction algorithm, which repeatedly satisfies requests from the currently available instances and marks those processes as able to finish. A deadlock exists only if some processes remain unmarked when no further request can be satisfied.

What the OS does not have is future requests, which is why detection is retrospective, run periodically or whenever a request would block, at $O(V + E)$ per check.

3. Scheduling

1. Uniprocessor scheduling

When FIFO is optimal for average response time

FIFO (First In, First Out) is optimal only in the special case where all jobs arrive at the same time and have the same length. In that case every ordering produces exactly the same average response time, because each job waits for the sum of the jobs ahead of it, and with equal lengths that sum is identical no matter who goes first. FIFO therefore cannot be beaten, but it also cannot beat anything. As soon as the jobs have different lengths, shortest job first is better, because running a short job first shortens the wait of many jobs at the cost of one long job. When jobs also arrive at different times, FIFO suffers from the convoy effect, where one long job at the head of the queue delays every short job behind it.

How MFQ combines FIFO, SJF and round robin

MFQ (Multilevel Feedback Queue) keeps several queues at different priority levels and serves them in priority order. A new process enters the top queue, which is served round robin with a small quantum, so interactive and short jobs get a fast response and share the CPU fairly. If a process uses up its whole quantum, it is demoted to the next lower queue, which has a larger quantum, so CPU-bound

jobs sink towards the bottom. The bottom queue is served FIFO, so a long job eventually finishes without being interrupted again. This approximates shortest job first (SJF), because a short job finishes while it is still in the top queue and never gets demoted, and it gives round robin fairness at the top and FIFO at the bottom. Aging is usually added, where a process is periodically promoted back to a higher queue, so that it cannot starve.

The shortcomings are that it is a heuristic with many parameters to tune, such as the number of queues, the quantum at each level, and the demotion and promotion rules, and there is no guarantee of fairness or optimality. It can also be gamed, because a process that yields just before its quantum expires keeps its high priority, and a process that mixes short I/O bursts with long CPU bursts can be misclassified. Without aging it starves low-priority processes, and with aging the promotion policy can undo the priority ordering it was trying to build.

2. Multi-core scheduling

Why a uniprocessor scheduler is inefficient on multicore

The first reason is lock contention. A uniprocessor scheduler keeps one global run queue, so every scheduling decision on every core must take the same lock, and the cores end up serializing on it instead of running in parallel. The second reason is lost cache affinity. A thread that is scheduled on a different core than last time finds its working set cold in that core's cache and its TLB (Translation Lookaside Buffer) entries gone, so it pays a migration cost for every move. The third reason is load imbalance. A single queue gives the scheduler one global view that is stale by the time it is used, so some cores sit idle while others have a backlog, and the scheduler itself becomes a serialization point that does not scale with the number of cores.

MFQ makes all three worse. Its several queues and their priority ordering are shared structures, so every enqueue, dequeue, and demotion takes a lock, and a thread demoted on one core while another core scans the same queue bounces that queue's cache line between cores. A thread that migrates also loses its place in the MFQ history, so its priority level no longer reflects how it has behaved, and the global priority order forces every core to consult the same structure before it can pick work.

Work stealing

Work stealing gives each core its own run queue, usually a deque, so the common case of a core taking work from its own queue needs no global lock and stays cache friendly. A core that runs out of work becomes a thief and takes a task from the tail of another core's deque, while the owner keeps taking from the head, which is the most recently created task and the one most likely to be warm in its cache. The victim's deque is the only shared structure touched, and only when a steal happens, so contention stays low and the design scales with the number of cores. The cost is that a stolen task is

cold in the thief's cache, and the deque needs its own synchronization so that the owner and the thieves do not corrupt it.